

使用 Vertex AI PaLM API 的文字摘要方法

來源：<https://codelabs.developers.google.com/text-summ-large-docs-stuffing?hl=zh-tw>

步驟數：7

在這個教學課程中，您將瞭解如何使用生成式模型，透過填充方法從文字中統整文字資訊

目錄

1. [1. 簡介](#)
2. [2. 需求條件](#)
3. [3. 費用](#)
4. [4. 開始使用](#)
5. [5. 填塞方法](#)
6. [6. MapReduce 方法](#)
7. [7. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

文字摘要是建立簡短版文字文件的程序，但仍會保留重要資訊。這項程序可用於快速重點瀏覽長篇文件、瞭解文章要點，或與使用者分享摘要。雖然摘錄短段落並非易事，但如果想摘錄大型文件，還需要克服一些挑戰。例如多頁 PDF 檔案。

在本程式碼研究室中，您將瞭解如何使用生成模型摘要大型文件。

建構項目

在本教學課程中，您將瞭解如何使用生成模型摘要文字資訊，方法如下：

- 填料
 - MapReduce
 - MapReduce with Overlapping Chunks
 - MapReduce with Rolling Summary
-

步驟 2 · 約 0 分鐘

2. 需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
 - 已啟用計費功能的 Google Cloud 雲端專案
-

步驟 3 · 約 0 分鐘

3. 費用

本教學課程使用 Vertex AI Generative AI Studio 做為 Google Cloud 的計費元件。

請參閱 [Vertex AI 的計價方式](#)和 [生成式 AI 的計價方式](#)，然後利用 [Pricing Calculator](#)，根據您預計的用量來預估費用。

4. 開始使用

1. 使用下列指令安裝 Vertex AI SDK、其他套件及其依附元件：



```
!pip install google-cloud-aiplatform PyPDF2 ratelimit backoff --upgrade --quiet --user
```

- 如果是 **Colab**，請取消註解下列儲存格，重新啟動核心。



```
# # Automatically restart kernel after installs so that your environment can access the new
packages
import IPython

app = IPython.Application.instance()
app.kernel.do_shutdown(True)
```

- 如果是 **Vertex AI Workbench**，可以使用頂端的按鈕重新啟動終端機。

2. 請透過下列任一方式驗證筆記本環境：

- 如果是 **Colab**，請取消註解下列儲存格。



```
from google.colab import auth
auth.authenticate_user()
```

- 如要使用 **Vertex AI Workbench**，請參閱[設定操作說明](#)。

3. 匯入程式庫，初始化 Vertex AI SDK。

- 如果是 **Colab**，請取消註解下列儲存格，匯入程式庫。

```
import vertexai

PROJECT_ID = "[your-project-id]" # @param {type:"string"}
vertexai.init(project=PROJECT_ID, location="us-central1")
```

```

import re
import urllib
import warnings
from pathlib import Path

import backoff
import pandas as pd
import PyPDF2
import ratelimit
from google.api_core import exceptions
from tqdm import tqdm
from vertexai.language_models import TextGenerationModel

warnings.filterwarnings("ignore")

```

4. 匯入模型，載入名為 text-bison@001 的預先訓練文字生成模型。

```

generation_model = TextGenerationModel.from_pretrained("text-bison@001")

```

5. 準備資料檔案，並下載 PDF 檔案以執行摘要工作。

```

# Define a folder to store the files
data_folder = "data"
Path(data_folder).mkdir(parents=True, exist_ok=True)

# Define a pdf link to download and place to store the download file
pdf_url =
"https://services.google.com/fh/files/misc/practitioners_guide_to_mlops_whitepaper.pdf"
pdf_file = Path(data_folder, pdf_url.split("/")[-1])

# Download the file using `urllib` library
urllib.request.urlretrieve(pdf_url, pdf_file)

```

以下說明如何查看下載的 PDF 檔案中的幾頁內容。

```

# Read the PDF file and create a list of pages
reader = PyPDF2.PdfReader(pdf_file)
pages = reader.pages

# Print three pages from the pdf
for i in range(3):
    text = pages[i].extract_text().strip()

print(f"Page {i}: {text} \n\n")

#text contains only the text from page 2

```

5. 填塞方法

將資料傳遞至語言模型最簡單的方式，就是將資料「塞入」提示做為背景資訊。包括提示中的所有相關資訊，以及您希望模型處理資訊的順序。

1. 只從 PDF 檔案的第 2 頁擷取文字。

```
# Entry string that contains the extracted text from page 2
print(f"There are {len(text)} characters in the second page of the pdf")
```

2. 建立提示範本，以便後續在筆記本中使用。

```
prompt_template = """
    Write a concise summary of the following text.
    Return your response in bullet points which covers the key points of the text.

    ```{text}```

 BULLET POINT SUMMARY:
 """
```

3. 透過 API 使用 LLM 匯總擷取的文字。請注意，大型語言模型目前有輸入文字限制，因此系統可能不接受大量輸入文字。如要進一步瞭解配額和限制，請參閱「[配額與限制](#)」。

下列程式碼會導致例外狀況。

```
Define the prompt using the prompt template
prompt = prompt_template.format(text=text)

Use the model to summarize the text using the prompt
summary = generation_model.predict(prompt=prompt, max_output_tokens=1024).text

print(summary)
```

4. 模型回應錯誤訊息：**400 要求包含無效引數**，因為擷取的文字過長，生成模型無法處理。

為避免發生這個問題，請輸入擷取文字的其中一部分，例如前 30,000 字。

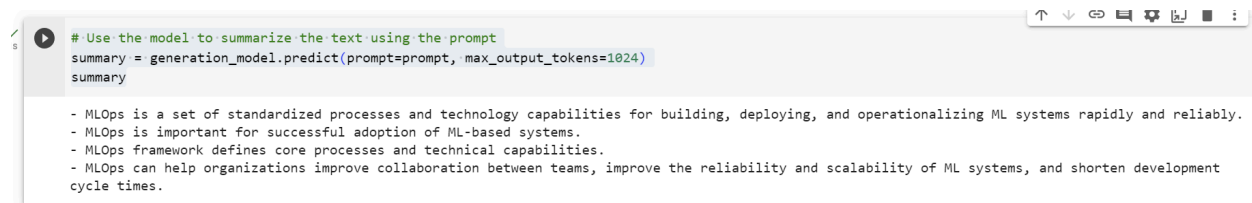
```
Define the prompt using the prompt template
prompt = prompt_template.format(text=text[:30000])

Use the model to summarize the text using the prompt
summary = generation_model.predict(prompt=prompt, max_output_tokens=1024)

summary
```

螢幕截圖應會顯示下列結果：





```
Use the model to summarize the text using the prompt
summary = generation_model.predict(prompt=prompt, max_output_tokens=1024)
summary

- MLOps is a set of standardized processes and technology capabilities for building, deploying, and operationalizing ML systems rapidly and reliably.
- MLOps is important for successful adoption of ML-based systems.
- MLOps framework defines core processes and technical capabilities.
- MLOps can help organizations improve collaboration between teams, improve the reliability and scalability of ML systems, and shorten development cycle times.
```

## 摘要

雖然模型無法處理完整文字，但您已使用模型，從 PDF 的一部分內容中，建立最重要的簡要項目符號清單。

### 優點

- 這個方法只會對模型發出一次呼叫。
- 在摘要文字時，模型會一次存取所有資料。這樣結果會更準確。

### 缺點

- 大多數模型都有上下文長度。如果文件很大 (或有很多文件)，這項功能就無法運作，因為產生的提示會超過脈絡長度。
  - 這個方法只適用於較小的資料片段，不適合用於大型文件。
-

步驟 6 · 約 0 分鐘

## 6. MapReduce 方法

為解決大型文件問題，我們將採用 MapReduce 方法。這項方法會先將大型資料分割成較小的片段，然後對每個片段執行提示。如果是摘要工作，第一個提示的輸出內容就是該段內容的摘要。生成所有初始輸出內容後，系統會執行不同的提示來合併這些內容。

如要瞭解這個方法的實作詳情，請參閱這個 [GitHub 連結](#)。

---

## 7. 恭喜

恭喜！您已成功統整長篇文件內容。您已學會 2 種長篇文件摘要方法，以及這些方法的優缺點。摘要大型文件的方法有幾種，請留意其他 2 種方法，也就是在另一個程式碼研究室中，使用重疊區塊的 MapReduce 和使用滾動摘要的 MapReduce。

摘要長篇文件可能很困難。您必須找出文件要點、統整資訊，並以簡潔連貫的方式呈現。如果文件內容複雜或涉及技術，這項作業可能會很困難。此外，總結長篇文件可能很耗時，因為您需要仔細閱讀和分析文字，確保摘要內容準確完整。

雖然這些方法可讓您以彈性方式與 LLM 互動，並摘要長篇文件，但有時您可能想使用自舉或預先建構的方法，加快處理速度。這時 LangChain 等程式庫就派上用場。進一步瞭解 [Vertex AI 的 LangChain 支援](#)。

---